

Android Compose核心标签详解：

@Composable及必备标签

上一讲我们认识了声明式UI、组合等核心思想，而这些思想在Jetpack Compose中落地的核心，就离不开 `@Composable` 标签。这一讲我们专门拆解这个"灵魂标签"，同时带你认识开发中必须掌握的其他重要标签。

一、@Composable：Compose的"界面入口"标签

先给小白一个最直白的定义：**@Composable是一个"标记"**，告诉Android系统：**这个函数是用来创建界面的**。就像给函数盖了一个"界面组件专用章"，系统看到这个章，就知道要按照声明式UI的规则来处理它。

1.1 为什么@Composable必不可少？

Compose的核心是"用函数描述界面"，但不是所有函数都能用来描述界面——只有被 `@Composable` 标记的函数才具备这个能力。我们通过对比来理解：

 没有@Composable的普通函数（无法用于界面）

Code block

```
1  // 普通函数：只能做数据处理，不能直接放在界面中
2  fun getGreeting(): String {
3      return "Hello Compose"
4  }
5
6  // 错误用法：普通函数不能嵌套界面组件
7  fun BadUI() {
8      // 这里写Text("你好")会直接报错，因为BadUI没有@Composable标记
9  }
10
```

 有@Composable的界面函数（可直接用于界面）

Code block

```
1  // 被@Composable标记的函数：专门用于创建界面
2  @Composable
3  fun GreetingUI() {
```

```
4      // Text本身也是被@Composable标记的系统组件
5      Text(text = getGreeting()) // 可以调用普通函数获取数据
6  }
7
```

 **核心原因：**@Composable会触发Compsole编译器的"特殊处理"——给函数添加上下文管理、重组追踪等能力，没有这个标记，函数就不具备界面组件的特性。

1.2 @Composable函数的3个关键规则

1. 参数可以是普通数据，也可以是其他@Composable函数 // 接收普通数据（字符串）

```
@Composable
fun NameTag(name: String) {
    Text(text = "姓名: $name")
}
```

// 接收另一个@Composable函数（content参数）

```
@Composable
fun CardView(content: @Composable () -> Unit) {
    Box(
        modifier = Modifier.border(1.dp, Color.Gray)
    ) {
        content() // 嵌入传入的界面组件
    }
}
```

2. **返回值必须是Unit（即没有返回值）** 因为@Composable函数的作用是"描述界面"，而不是"返回数据"，所以不能写返回值类型（默认就是Unit）。 `// 错误：不能有返回值`

```
@Composable
fun BadFunction(): String {
    return "错误"
}
```

`// 正确：无返回值`

```
@Composable
fun GoodFunction() {
    Text(text = "正确")
}
```

3. **只能在@Composable函数中调用**就像"只有在厨房才能做饭"，被@Composable标记的函数（包括系统组件如Text、Button），只能在其他@Composable函数里使用。 `// 正确：在@Composable函数中调用Text（系统@Composable组件）`

```
@Composable
fun MyScreen() {
    Button(onClick = { /* 点击事件 */ }) {
        Text(text = "点击我") // 正确调用
    }
}
```

```
// 错误：在普通函数中调用@Composable组件
fun 普通Function() {
    Text(text = "错误") // 直接报错
}
```

二、除了@Composable，你还必须掌握这些标签

@Composable是基础，但实际开发中需要配合其他标签来完成数据管理、状态监听等核心功能，这些标签都是Compose的"标配工具"。

2.1 remember：让数据"记住"状态（解决重组丢失问题）

问题场景：重组导致数据重置

上一讲我们提到"重组"——数据变了界面会重新绘制。但如果数据没有被特殊处理，重组时会被重置：

```

1 @Composable
2 fun Counter() {
3     // 普通变量：每次重组都会重新变成0
4     var count = 0
5
6     Button(onClick = { count++ }) {
7         Text(text = "点击了$count次")
8     }
9 }
10 // 现象：点击按钮后，数字不会增加，因为重组时count又变回0了
11

```

解决方案：用remember标记状态变量

`remember` 的作用是"在重组时保留数据"，相当于给数据找了一个"安全的仓库"，重组时不会被清空。

Code block

```

1 @Composable
2 fun Counter() {
3     // remember标记：重组时保留count的值
4     var count by remember { mutableStateOf(0) }
5
6     Button(onClick = { count++ }) {
7         Text(text = "点击了$count次")
8     }
9 }
10 // 现象：点击按钮，数字正常增加，因为count被remember保护了
11

```



`mutableStateOf(0)`：用来创建"可观察的状态"——当count的值变化时，会自动触发使用count的组件重组（也就是Text会重新绘制）。`remember`和`mutableStateOf`通常是"搭档"。

2.2 @Preview：界面预览标签（不用运行APP也能看效果）

开发时总不能每次改一点界面就运行APP看吧？`@Preview` 就是用来在Android Studio中直接预览界面的"神器"，它标记的`@Composable`函数会在预览窗口显示效果。

基础用法：简单预览

Code block

```
1 // 1. 要预览的界面组件
2 @Composable
3 fun MyPreviewUI() {
4     Column {
5         Text(text = "预览测试", fontSize = 20.sp)
6         Button(onClick = {}) {
7             Text(text = "预览按钮")
8         }
9     }
10 }
11
12 // 2. 预览函数：用@Preview标记
13 @Preview(showSystemUi = true, name = "我的界面预览")
14 @Composable
15 fun PreviewMyUI() {
16     // 直接调用要预览的组件
17     MyPreviewUI()
18 }
19
```

@Preview的常用参数

参数	作用
name	给预览起名字，多个预览时方便区分
showSystemUi	是否显示系统状态栏和导航栏，true更贴近真实效果
device	指定预览的设备型号，比如"pixel_5"
fontScale	测试字体缩放（比如1.5f模拟大字体模式）

2.3 LaunchedEffect：在Compose中启动协程

Android中网络请求、延时操作等耗时任务需要用协程（避免卡界面），但普通协程不能直接在@Composable函数中启动——必须用 `LaunchedEffect`。

核心作用

1. 关联组件的生命周期：组件显示时启动协程，组件消失时自动取消协程（避免内存泄漏）；2. 可通过"键"控制协程是否重新启动。

实际案例：界面显示后延迟2秒提示

Code block

```
1  @Composable
2  fun DelayTip() {
3      var tip by remember { mutableStateOf("") }
4
5      // 启动协程: key为Unit时, 组件首次显示只执行一次
6      LaunchedEffect(key1 = Unit) {
7          // 延迟2秒 (协程的延时函数)
8          delay(2000)
9          // 修改状态, 触发重组
10         tip = "界面已经显示2秒啦! "
11     }
12
13     Text(text = tip)
14 }
15
```

2.4 SideEffect：处理界面副作用

"副作用"指的是"不直接参与界面描述的操作", 比如修改全局变量、上报统计数据、更新系统状态栏颜色等。这些操作不能直接写在@Composable函数里 (可能导致多次执行), 需要用 `SideEffect` 包裹。

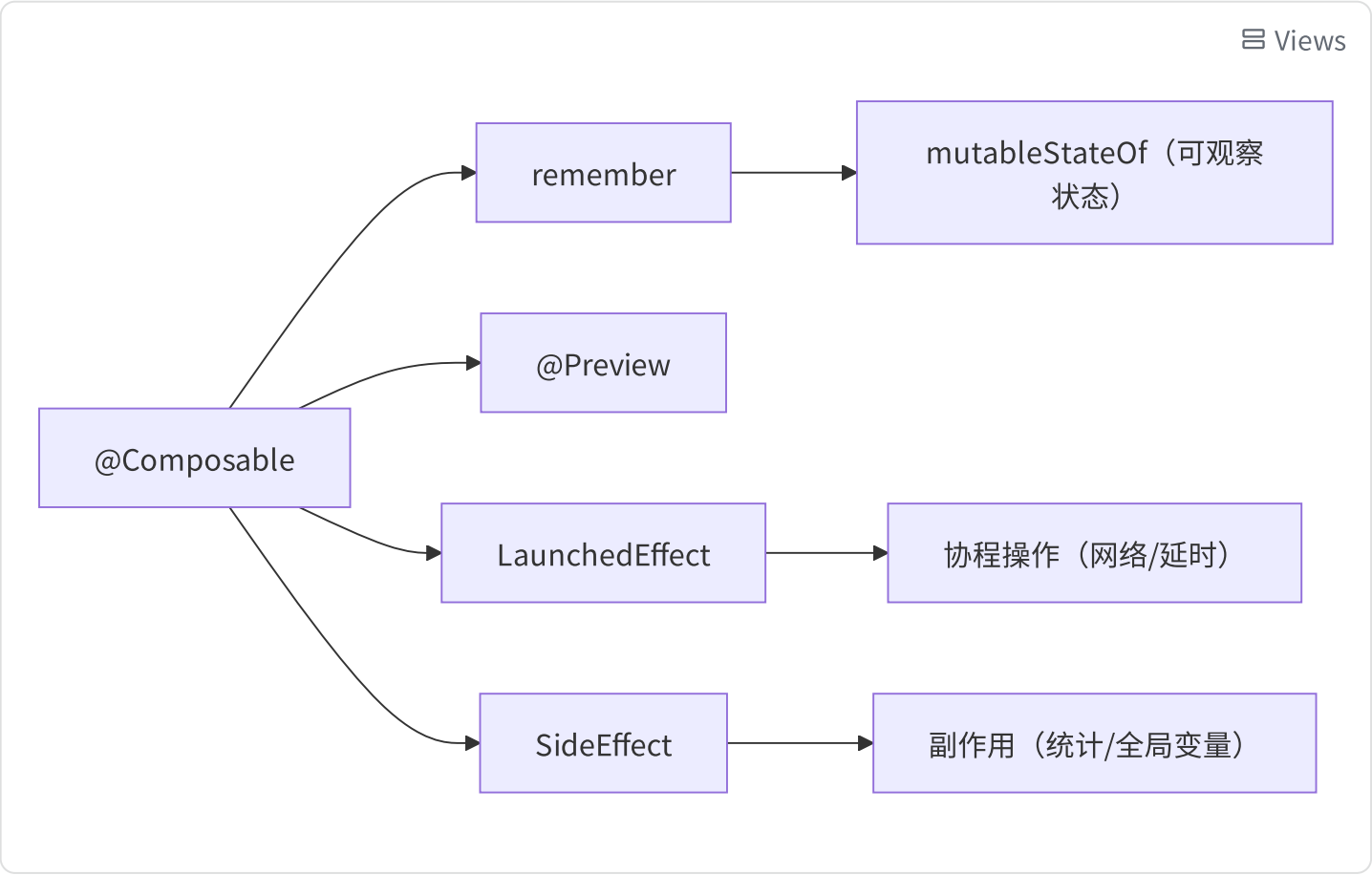
案例：界面重组时上报数据

Code block

```
1  @Composable
2  fun ReportScreen() {
3      var count by remember { mutableStateOf(0) }
4
5      // 副作用操作: 每次重组都会执行
6      SideEffect {
7          // 上报当前count值 (模拟统计)
8          Analytics.report("count", count)
9      }
10
11     Button(onClick = { count++ }) {
12         Text(text = "count: $count")
13     }
14 }
15
```

三、核心标签关系图与使用场景总结

3.1 标签关系速记



3.2 场景对应表

开发场景	必须用的标签
创建任何界面组件	@Composable
保存界面状态（如计数、输入框内容）	remember + mutableStateOf
在Android Studio中看界面效果	@Preview
执行耗时操作（网络请求、延时）	LaunchedEffect
执行统计、修改全局变量等操作	SideEffect

四、实战小练习：组合所有标签

需求：做一个"点击计数器"，包含以下功能：1. 显示点击次数；2. 点击按钮计数+1；3. 界面显示后2秒提示"开始点击吧"；4. 支持预览。

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Button
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.*
5  import androidx.compose.ui.tooling.preview.Preview
6  import kotlinx.coroutines.delay
7
8  // 1. 核心界面组件 (@Composable标记)
9  @Composable
10 fun ClickCounter() {
11     // 2. 用remember保存状态, mutableStateOf让状态可观察
12     var count by remember { mutableStateOf(0) }
13     var tip by remember { mutableStateOf("") }
14
15     // 3. 用LaunchedEffect启动协程 (延时提示)
16     LaunchedEffect(key1 = Unit) {
17         delay(2000)
18         tip = "开始点击吧! "
19     }
20
21     // 4. 界面布局
22     Column {
23         Text(text = tip, fontSize = 18.sp)
24         Text(text = "已点击: $count 次", fontSize = 20.sp)
25         Button(onClick = { count++ }) {
26             Text(text = "点我+1")
27         }
28     }
29 }
30
31 // 5. 预览函数 (@Preview标记)
32 @Preview(showSystemUi = true, name = "计数器预览")
33 @Composable
34 fun PreviewClickCounter() {
35     ClickCounter()
36 }
37
```



把这段代码复制到Android Studio的Compose项目中，就能直接看到预览效果，点击按钮也能正常计数——这就是核心标签的协同作用！

五、重点回顾

- @Composable：界面组件的"身份证"，必须有才能创建界面，且只能在同类函数中调用；
- remember：状态的"保险柜"，防止重组时数据丢失，配合mutableStateOf实现状态监听；
- @Preview：开发效率"加速器"，不用运行APP就能看界面；
- LaunchedEffect/SideEffect：分别处理协程任务和副作用，避免内存泄漏和异常执行。

这些标签是Compose开发的基础，先把它们的用法记牢，后续复杂界面都是基于这些标签的组合扩展

~

(注：文档部分内容可能由 AI 生成)